AppliedMath

MDPI

Article

gp3sequences: An Auditable and Reproducible Framework for Ordered Categorical Sequence Analysis in R

Stefanos Balaskas

eGovernment & eCommerce Lab (Innovation & Entrepreneurship), Department of Business Administration, University of Patras, 26504 Patras, Greece; s.balaskas@ac.upatras.gr

Abstract

Ordered categorical sequence analysis is distributed across specialist methods and software, making preparation decisions, method settings, diagnostics, and provenance difficult to inspect consistently across an end-to-end workflow. This paper presents gp3sequences, an R framework that adds a domain-specific contract layer to reproducible computing: explicit long-format mappings, declared preparation policies and decision logs, structured analysis contracts, capability-aware specialist hand-offs, and machine-readable provenance. The framework does not replace specialist implementations. Evaluation combines a controlled four-state Markov benchmark, independent numerical-reference checks, adversarial failure cases, multi-seed and design-size sensitivity analyses, a bounded scaling experiment, and a public six-state TraMineR 2.2-14 data example for interoperability. The primary benchmark contains 72 sequences of length 24 and deliberately overlapping transition structure. Review-level validation diagnostics are shown to arise from preserved consecutive repeated states rather than data errors. The prespecified Levenshtein two-cluster solution is weakly separated, and sensitivity analyses confirm that clustering conclusions depend on the selected sequence representation; the result is therefore treated as a diagnostic demonstration rather than evidence of latent classes. Across the evaluated operations, gp3sequences preserves declared analytical state while exposing invalid mappings, unavailable dependencies, and other structural problems rather than silently resolving them. The contribution is accordingly an integration and governance architecture for inspectable ordered-sequence workflows, not a claim of methodological or computational superiority over specialist packages.

Keywords: ordered categorical sequences; sequence analysis; scientific software; reproducibility; provenance; interoperability; R; software validation; auditability

Check for updates

Received: 3 September 2026
Revised: 23 September 2026
Accepted: 24 September 2026
Published: 9 October 2026

Copyright: © 2026 by the author. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Ordered categorical sequences represent trajectories in which the order of discrete states is analytically relevant. Examples include life-course histories, event sequences, interaction traces, navigation paths, educational activities, and scanpath-like categorical paths. Unlike a cross-sectional frequency table, a sequence retains temporal or positional structure: identical state totals can coexist with materially different ordering, recurrence, persistence, and transition patterns. Sequence analysis therefore spans descriptive summaries, dissimilarity measures, clustering, frequent-pattern extraction, Markov representations, and latent-state models rather than a single inferential family [1–4]. The methodological breadth is productive but creates a computational consequence. Different questions are naturally addressed through different representations. State-sequence methods emphasise whole-path

structure; sequential-pattern methods emphasise recurring ordered fragments; Markov models emphasise transition dependence; hidden Markov models distinguish observed categorical symbols from latent model states; and cluster validation concerns properties of a derived partition rather than the original sequences [5–7]. A realistic analysis may use several of these perspectives on the same data.

The R ecosystem reflects this methodological specialisation. TraMineR provides a mature environment for defining, describing, visualising, and comparing state sequences and for computing sequence dissimilarities [2]. seqHMM focuses on hidden and mixture hidden Markov models, including multichannel sequence settings [8]. arulesSequences provides frequent sequential-pattern mining through cSPADE, whose algorithmic foundation is SPADE [5,9]. WeightedCluster adds weighted clustering, cluster-quality assessment, and typology validation [10,11]. ClickClust addresses finite-mixture Markov clustering [12], PST implements probabilistic suffix-tree analysis for variable-memory sequence structure [13], and march provides a family of Markovian models for categorical data [14,15]. This division of labour should not be treated as a software defect. Specialist packages encode method-specific assumptions and object semantics that would be counterproductive to flatten into one universal representation. The integration problem arises instead when an applied workflow crosses those boundaries. Preprocessing choices may be resolved in one object, dissimilarities in another, clustering diagnostics elsewhere, and model-specific settings in a fourth representation. Without deliberate controls, a researcher can reproduce the numerical algorithm while still losing the provenance of data mappings, filtering decisions, state ordering, random seeds, design declarations, or optional backends.

Reproducibility is not secured by code availability alone. Reproducible computational research requires sufficiently specified data, transformations, software environments, and analytical decisions for a result to be reconstructed and inspected [16,17]. Robust research software likewise benefits from explicit testing, small auditable components, dependency control, and failure modes that expose rather than conceal invalid states [18]. Research compendia and FAIR principles extend this logic to the packaging, discoverability, and reuse of computational artefacts [19,20]. For sequence analysis, four forms of analytical state are particularly easy to lose. First, data mappings determine which columns constitute sequence identity, within-sequence order, state, duration, grouping, or analytical unit. Second, preparation policies determine how missing states, duplicated positions, repeated states, and undeclared symbols are handled. Third, method settings determine distances, normalisation, cluster specification, model order, and resampling. Fourth, interpretation depends on study design: a cluster is not automatically a behavioural type, a latent HMM state is not automatically a cognitive state, and a structural group difference is not automatically causal. An auditable workflow should keep these distinctions available after the numerical result has been computed.

gp3sequences implements one response to this integration problem [21]. Its primary boundary is an ordinary long-format data frame rather than a new universal specialist object. Validation and preparation make data mappings and structural policies explicit; downstream modules operate on canonical sequence data; analysis contracts retain recoverable settings, identifiers, diagnostics, capabilities, seeds, and provenance; and adapters delegate to specialist software when specialist semantics are required. This architecture should be distinguished from general reproducible-workflow practice. R Markdown or Quarto can preserve executable code and narrative, research compendia can package data and dependencies, and wrappers can simplify function calls. gp3sequences instead enforces sequence-specific runtime contracts: it validates ordered-state structure, records preparation policies and affected rows, exposes capability and backend status, attaches audit metadata to analytical objects, and prevents several ambiguous or unsupported

operations from proceeding silently. These mechanisms are complementary to, rather than substitutes for, literate programming, dependency management, or specialist statistical packages [4,21].

Against this background, the unresolved problem is not a shortage of sequence-analysis methods, but the absence of a common, inspectable software contract when a workflow crosses representations and specialist tools. The present study therefore evaluates whether gp3sequences can provide that contract while leaving method-specific estimation and interpretation with the relevant analytical family.

The study makes four restrained contributions. First, it evaluates a sequence-data contract that separates identity, order, state, metadata, and preparation policies from downstream method choice. Second, it evaluates analysis contracts and reference checks that retain or reconstruct settings, diagnostics, identifiers, seeds, and provenance across representative structural families. Third, it formalises guarded interoperability as delegation and demonstrates a concrete TraMineR hand-off, while preserving specialist methodological ownership. Fourth, it evaluates the architecture using controlled success, sensitivity, empirical interoperability, and failure cases rather than a single favourable execution. The evaluation is organised around five research questions:

RQ1. Can a stable long-format sequence-data contract support heterogeneous ordered categorical sequence analyses while keeping preprocessing decisions explicit?

RQ2. Can structured analysis contracts preserve inspectable settings, diagnostics, identifiers, and provenance across representative sequence-analysis families?

RQ3. Can controlled benchmark and sensitivity analyses recover deliberately introduced structural characteristics without implying stronger substantive or clustering conclusions than the selected representation supports?

RQ4. How does the framework complement established specialist R software rather than duplicate its methodological depth?

RQ5. How does computational cost change across a bounded range of sequence counts and sequence lengths for validation, preparation, state summaries, and transition summaries?

The remainder of the paper is organised as follows. Section 2 describes the design requirements, contract architecture, verification strategy, benchmark, sensitivity analyses, scalability experiment, and public-data interoperability example. Section 3 reports the validation, structural analyses, reference checks, adversarial tests, sensitivity results, and empirical hand-off. Section 4 discusses the scientific-software contribution, its relationship to specialist software, reproducibility, and interpretation governance. Section 5 states the limitations, and Section 6 concludes.

2. Materials and Methods

2.1. Design Requirements

Five design requirements define the evaluation. R1 (neutral input): ordinary long-format data should remain the stable boundary between raw observations and analytical methods. R2 (explicit preparation): structural irregularities should be resolved through declared policies rather than silent destructive preprocessing. R3 (deterministic computation): stochastic procedures should expose and record seeds or otherwise document their randomness. R4 (auditable results): analysis objects should retain settings, diagnostics, identifiers, capabilities, and provenance where technically appropriate. R5 (bounded interpretation): structural outputs should remain distinct from psychological, behavioural, diagnostic, or causal claims unless external evidence and study design justify those interpretations.

These requirements reflect established reproducibility and research-software principles while being evaluated here within a sequence-analysis setting [16–18].

2.2. Software Architecture

The architecture separates the persistent data boundary from method-specific computation. Validation checks identity, ordering, coding, metadata, and state declarations; preparation applies only declared policies and returns a decision log; analytical modules consume the canonical representation; and analysis contracts preserve information required for later audit. Table 1 makes the framework-specific contribution explicit relative to a conventional reproducible R workflow. The distinction is enforcement rather than document generation: a literate workflow can record arbitrary code, whereas the package contract constrains sequence-specific inputs, records policy choices in machine-readable form, and exposes unsupported operations or missing backends. A schematic of the architecture is retained as Appendix A Figure A1 to avoid duplicating the information carried by Table 1.

Table 1. Sequence-specific contract features relative to a conventional reproducible R workflow.

Concern	Conventional Reproducible R Workflow	gp3sequences Contract-Oriented Addition
Input semantics	Column roles and ordering conventions are encoded in analyst code.	Explicit sequence-ID, order, state, duration, and metadata mappings are validated before analysis.
Preparation	Transformations can be reproducible but policy choices may remain implicit in code.	Missing, duplicate, repeated, zero-duration, and undeclared-state policies are declared; affected rows and decisions are returned.
Analytical state	Settings may be dispersed across heterogeneous package objects and prose.	Supported results expose family, method, identifiers, state levels, settings, seed status, diagnostics, and provenance for audit.
Capabilities and dependencies	Availability is commonly handled by ad hoc checks or installation instructions.	Capability reporting and guarded adapters expose backend availability; optional packages are not silently installed or loaded.
Cross-family comparison	Manual bookkeeping is required to establish whether two outputs are semantically comparable.	Contract comparison exposes family/settings/provenance differences without silently coercing distinct analytical families.
Failure handling	Executable code can still reproduce an undocumented or silent preprocessing choice.	Structural problems receive machine-readable severity/issue codes or explicit stop conditions; preparation does not silently repair fatal ambiguity.

2.3. Sequence-Data Contract

The primary contract requires configurable columns for sequence identity, within-sequence order, and categorical state. Participant, trial, condition, duration, weight, channel, analytical-unit, and other metadata can remain attached when they are structurally compatible. The contract is intentionally conservative. It does not assume that a row is valid simply because the columns exist: ordering, duplicate positions, missing or unknown states, and metadata consistency are checked before analysis.

audit_sequence_data() reports machine-readable issues without modifying the input. validate_sequence_data() reduces the audit to a validity/status contract, and prepare_sequence_data() applies explicit policies while retaining mapping, audit, and decision records. The primary benchmark uses refusal policies for missing states, duplicated positions, and undeclared symbols and preserves consecutive repeated states. A review status therefore does not imply invalid data: review diagnostics identify structures requiring an explicit decision, whereas error diagnostics block preparation unless a documented corrective policy is available.

2.4. Analysis Contracts and Provenance

Method-specific outputs retain more than their principal numerical result. The contract layer records or reconstructs the analytical family, method, package and contract versions, state levels, sequence identifiers, settings, backend information, seed status, diagnostics, and provenance where technically supported. audit_sequence_analysis() inspects supported results against that contract, and compare_sequence_analysis_results() compares recoverable analytical state without silently treating different families as equivalent. The revised evaluation audits distance, clustering, and first-order transition-network objects; repeated distance computation remains a deterministic regression check rather than the sole correctness criterion.

This design treats provenance as part of the analytical object rather than a narrative added after computation. It cannot guarantee scientific validity: a perfectly recorded inappropriate method remains inappropriate. Its narrower purpose is to make analytical state available for inspection.

2.5. Analytical Modules

The package groups methods by analytical purpose rather than by a single universal sequence representation. State and transition summaries describe observed categorical structure. Contiguous n-grams and bounded non-contiguous subsequences describe recurring local patterns. Dissimilarity methods represent whole-sequence differences and can feed clustering, representative-sequence, or stability analyses. Transition networks and higher-order models represent recent-state dependence. Categorical, mixture, multichannel, and covariate-dependent HMM workflows address latent-state formulations whose interpretation remains model-based [6,8]. Design-aware permutation and bootstrap procedures require the independent unit and design to be declared before group inference. Visualisation functions expose these structural and diagnostic outputs without assigning substantive meanings to state labels.

The benchmark uses only a subset of this breadth. The evaluation therefore focuses on representative cross-family operations rather than exhaustive API coverage.

2.6. Guarded Interoperability

Interoperability is implemented as explicit delegation. Optional packages remain optional and are neither installed nor loaded silently. For TraMineR, as_traminer_sequences() converts the canonical long representation to a state-sequence object using the declared alphabet and sets the external object row names to the original sequence identifiers. The corresponding seqHMM adapter delegates through the TraMineR representation after explicitly checking both dependencies. The empirical worked example in Section 2.13 verifies identifier, state-level, ordering, and metadata consistency at this boundary; the purpose is to test the adapter contract, not numerical superiority or object equivalence.

2.7. Comparator-Software Evidence Protocol

Comparator packages were included when they represent a documented specialist family that gp3sequences either delegates to, overlaps structurally with, or must distinguish from its integration role: state-sequence analysis (TraMineR), latent-state modelling (seqHMM), sequential-pattern mining (arulesSequences), clustering and typology validation (WeightedCluster), finite-mixture Markov clustering (ClickClust), variable-memory modelling (PST), and Markov-family modelling (march). Evidence was taken from peer-reviewed software papers where available and current official manuals or CRAN records otherwise. Absence from the reviewed documentation is not interpreted as proof that a capability is impossible in another version.

The comparison is intentionally scope-qualified rather than a feature-count or performance ranking. Claims are limited to each package's documented specialist role and to the explicit gp3sequences boundary. TraMineR, seqHMM, ClickClust, and PST are supported by their software papers; arulesSequences, WeightedCluster, and march are additionally checked against their current package documentation. This evidence protocol is separate from the numerical-reference checks described below [5,8–15].

2.8. Software Verification and Validation

The executable evaluation uses five validation layers. First, package/API gates stop execution when required functions or versions are unavailable. Second, structural assertions verify benchmark dimensions, canonical prepared data, pair counts, and resampling settings. Third, audit_sequence_analysis() checks supported distance, clustering, and transition-network objects. Fourth, representative numerical outputs are recomputed through independent or lower-level references: selected edit distances via base-R edit distance, motif frequencies via explicit sliding-window counts, transition probabilities via contingency-table normalisation, hierarchical clustering via direct stats::hclust()/cutree(), silhouette values via cluster::silhouette() when available, and permutation/bootstrap summaries directly from the stored resampling distributions. Fifth, explicit adversarial fixtures test structural failures. Equality checks use a numerical tolerance of 1×10^{-12} where floating-point comparison is required.

These checks complement rather than replace package-level unit, contract, metamorphic, adversarial, documentation, and release validation. Determinism and numerical correctness are therefore treated as distinct properties: rerunning the same operation verifies repeatability, whereas the reference calculations verify selected numerical semantics against an independently expressed calculation or the underlying specialist/base implementation [17,18].

2.9. Synthetic Benchmark Design

The primary benchmark contains 72 sequences of 24 positions (1728 long-format rows), a four-state alphabet {A, B, C, D}, and two neutral groups generated by first-order Markov processes. These dimensions were selected to keep every pairwise operation and figure executable within the manuscript while still yielding 2556 sequence pairs and 1656 observed transitions. Four states provide a non-binary alphabet in which recurrence, self-transition, cross-state transition, motif, and edit-distance behaviour remain visually inspectable. Two groups are sufficient to exercise the declared-design inference API without implying a broader factorial design. First-order Markov generation provides known local transition structure while retaining stochastic paths and overlapping support; it is a validation device, not a claim that empirical sequences are first-order Markov.

The primary realisation uses seed 20260811 to make all reported tables and figures exactly reproducible. To separate reproducibility from seed-specific recovery, the revision adds 100 independently seeded replications of the same generator and a small dimension grid varying sequence count and length. The six-state public example in Section 2.13 provides an additional alphabet-size check. These analyses assess whether the qualitative recovery and contract behaviour depend on a single favourable draw; they do not establish universal statistical power across arbitrary transition structures.

2.10. Benchmark Pipeline

The benchmark executes one declared pipeline: data audit and validation; explicit preparation; state and transition summaries; extraction of contiguous two- and three-state motifs; extraction of length-two bounded subsequences with maximum gap one and maximum span three; normalised Levenshtein dissimilarity; average-linkage hierarchical

clustering with a prespecified two-cluster description; cluster validation; first-order transition networks; sequence-level state-A comparison under an observational design; bootstrap uncertainty; and analysis-contract auditing. Levenshtein dissimilarity is normalised by maximum sequence length. The clustering stage is descriptive and is evaluated using silhouette and Dunn diagnostics rather than treated as a classifier [3,7].

The group contrast declares analysis_unit—an explicit copy of sequence identity—as the independent-unit field. Sequence-level state-A prevalence is first computed for each unit. For each of 999 permutation iterations, complete group labels are permuted across the declared independent units without replacement while preserving the original group counts; the two-sided *p*-value is (1 + number of permuted absolute contrasts at least as large as the observed absolute contrast)/(999 + 1). For the bootstrap interval, independent units are resampled with replacement within each group, each bootstrap sample retains the original number of units per group, and the group-mean difference is recomputed. The reported 95% interval is the percentile interval from 999 bootstrap estimates using the package's declared quantile convention. Both procedures use seed 20260811. Because the synthetic group label is not a randomised treatment, the contrast remains associational.

2.11. Computational Scalability Experiment

A bounded scalability experiment evaluates only validation, preparation, state summarisation, and transition summarisation over 100, 500, and 1000 sequences crossed with lengths 10, 25, and 50. RQ5 is intentionally restricted to these core operations. Pairwise dissimilarity, clustering, pattern combinatorics, latent-state fitting, optional backends, and memory consumption are outside this scaling claim because their complexity and implementation dependencies require separate method-specific benchmarks.

Each operation is timed 3 times with system.time(), and the median elapsed time is retained. Timings are therefore environment-specific descriptive evidence, not portable performance guarantees. No comparison with specialist packages is attempted because computational equivalence would require carefully aligned data structures, algorithms, costs, and normalisation semantics.

2.12. Sensitivity, Reference-Equivalence, and Adversarial Checks

Seed sensitivity repeats the primary generator 100 times with seeds 20260811 + 1:100. Dimension sensitivity evaluates smaller/shorter and larger/longer designs around the primary setting; interpretation is limited to directional recovery and dispersion rather than formal power. Clustering sensitivity varies k = 2–5, hierarchical linkage, and sequence representation (Levenshtein, LCS, and transition-profile distance). An 80% sequence-subsampling stability analysis uses 200 resamples. Adversarial fixtures independently introduce a duplicated position, a missing state, an undeclared symbol, inconsistent within-sequence metadata, and an unavailable optional backend. Each fixture has a prespecified expected diagnostic or stop condition, so failure behaviour is tested rather than merely described.

2.13. Public Empirical Interoperability Example

A secondary worked example uses the public mvad data distributed with TraMineR and derived from McVicar and Anyadike-Danes [22]. The dataset contains 712 individuals, 72 monthly labour-market states, six documented states, identifiers, sample weights, and background covariates. It is reshaped to the gp3sequences long contract and used only to test realistic mapping, metadata retention, and specialist hand-off. The example verifies that sequence identifiers and the six-state alphabet are preserved when converting to a TraMineR state-sequence object and that metadata remain attached to the canonical long data. No substantive re-analysis of youth labour-market outcomes is attempted.

3. Results

3.1. Implemented Architecture and Analytical Coverage

Table 1 distinguishes the package contribution from generic reproducibility infrastructure. The framework does not claim novelty for executable documents, dependency recording, or research compendia. Its contribution is the sequence-specific enforcement layer that persists mappings, policy decisions, analytical settings, capability state, and diagnostic/provenance fields across heterogeneous operations.

The active sequence_capabilities() call returned 18 capability records for the current software environment. This inventory is environment-sensitive because optional backends may or may not be installed; the manuscript therefore reports the detected software state rather than assuming a fixed library configuration.

3.2. Validation and Explicit Preparation

The generated benchmark contained 1728 observations nested in 72 sequences. Validation returned review status with 0 error-level and 471 review-level diagnostics; preparation retained all 1728 rows. Table 2 resolves the apparently large review count: all 471 review records are consecutive_repeated_state flags. Such repeats are expected when a Markov path self-transitions and were deliberately preserved. No duplicated positions, missing states, inconsistent metadata, or undeclared states were present. Accordingly, the review status records an explicit modelling decision rather than 471 invalid observations.

Table 2. Diagnostic frequencies, severity, affected unit, and preparation consequence for the primary benchmark.

Diagnostic Code/Outcome	Severity	Affected Unit	Count	Interpretation in Primary Benchmark
consecutive_repeated_state	review	row within sequence	471	Expected self-transitions under the Markov generator; preserved by the declared repeated-state policy.
duplicated_position	error	sequence-position	0	No duplicate sequence positions.
missing_state	error	row	0	No missing state values.
inconsistent_metadata	error	sequence	0	Declared metadata are constant within sequence.
unknown_state	review at audit; refusal under selected policy	row	0	No states outside {A,B,C,D}.
All error-level diagnostics	error	various	0	No unresolved structural error.
Rows retained after preparation	—	row	1728/1728	Review flags do not trigger deletion; no row was silently removed.

These results address RQ1 at the level of a controlled software evaluation. The same long-format boundary supported validation, preparation, descriptive summaries, pattern extraction, distance analysis, network construction, and design-aware inference without method-specific input objects. The result is not evidence that empirical irregularities can be resolved generically; the adversarial and public-data examples below show where explicit decisions or refusal are required.

3.3. Position Relative to Specialist Software

Table 3 reports the inclusion logic, reviewed versions, specialist roles, and evidence date. It is not an accuracy, completeness, or feature-count ranking. The comparison sup-

ports RQ4 only in the narrower sense that gp3sequences occupies a governance/integration boundary that is different from the documented specialist focus of the comparator packages.

Table 3. Scope-qualified positioning of gp3sequences relative to selected specialist R software.

Package (Version Reviewed)	Primary Documented Specialist Role	Relationship/Boundary	Evidence and Access Date
gp3sequences 0.3.0	Cross-family validation, preparation, structural/inferential workflows, contracts and audit	Focal integration/governance layer; not a replacement for specialist depth	CRAN manual; tagged source v0.3.0; 21 Sep 2026
TraMineR 2.2-14	State-sequence representation, description, visualisation and dissimilarities	Specialist state-sequence complement and direct adapter target	Gabadinho et al. [2]; official/CRAN documentation; 21 Sep 2026
seqHMM 2.2.0	Categorical, mixture and multichannel hidden Markov models	Specialist latent-state complement	Helske and Helske [8]; official/CRAN documentation; 21 Sep 2026
arulesSequences 0.2-32	Frequent sequential-pattern mining with cSPADE	Specialist pattern-mining complement	Buchta and Hahsler [9]; package documentation; 21 Sep 2026
WeightedCluster 2.0	Weighted clustering, cluster quality and typology validation	Specialist clustering/validation complement	Studer [10]; package documentation; 21 Sep 2026
ClickClust 1.1.6	Finite-mixture Markov clustering of categorical sequences	Specialist model-based clustering complement	Melnykov [12]; package documentation; 21 Sep 2026
PST 0.94.1 (archived)	Probabilistic suffix trees and variable-memory Markov modelling	Specialist variable-memory complement	Gabadinho and Ritschard [13]; archived CRAN source; 21 Sep 2026
march 3.3.2	Markov-chain, MTD, HMM and double-chain models	Specialist Markov-family complement	Berchtold et al. [14]; package documentation; 21 Sep 2026

The comparison supports RQ4 by identifying complementarity rather than redundancy. TraMineR remains a specialist state-sequence environment [2]; seqHMM remains a specialist latent-state and mixture-model environment [8]; and the remaining comparators provide dedicated pattern-mining, clustering, variable-memory, or Markov-family interfaces [9,10,12–14]. The focal framework's narrower role is to preserve explicit preparation and provenance when an analysis traverses more than one such family.

3.4. Recovery of Known Synthetic Structure

The two generating processes had overlapping state support but different transition tendencies. The observed state distributions retained that intended partial overlap rather than producing mutually exclusive state sets. Figure 1 reports the realised observation proportions by state and group.

Transition summaries were generated from the same canonical data, including self-transitions. Because the processes share a common alphabet and all transition probabilities are positive, group differences are interpreted as structural tendencies rather than deterministic signatures. This is important for RQ3: the benchmark tests whether multiple analytical families recover a common known contrast without defining group membership through a single perfectly discriminating sequence pattern.

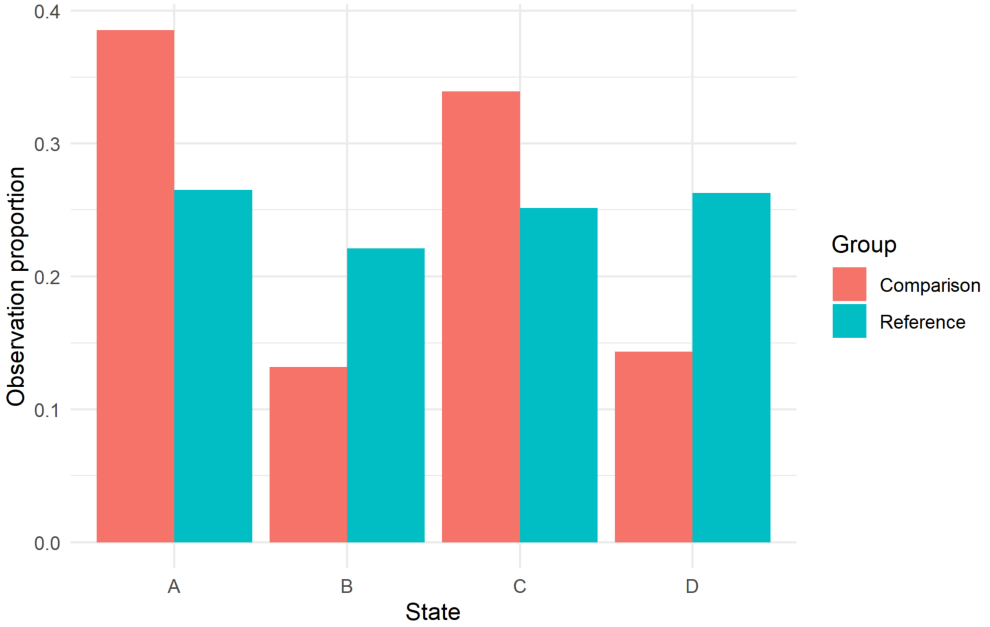


Figure 1. Realised state distributions in the primary synthetic benchmark. Both groups retain all four states; differences arise from the generating transition processes rather than exclusive state labels.

3.5. Motifs and Bounded Subsequences

Contiguous and bounded pattern extraction returned recurring structures in both groups. Table 4 reports the three most prevalent motifs and bounded subsequences per group, ranked within pattern family by sequence prevalence and then occurrence count. Full pattern inventories are generated in memory and are intentionally not printed in the article.

Table 4. Representative contiguous motifs and bounded non-contiguous subsequences generated from the synthetic benchmark.

Pattern_Type	Group	Pattern	Length	Occurrences	Sequence Prevalence
Contiguous motif	Comparison	C > A	2	126	1.000
Contiguous motif	Comparison	A > C	2	125	0.972
Contiguous motif	Comparison	A > A	2	129	0.944
Contiguous motif	Reference	C > D	2	95	0.972
Contiguous motif	Reference	A > B	2	80	0.972
Contiguous motif	Reference	D > A	2	89	0.917
Bounded subsequence	Comparison	A > A	2	249	1.000
Bounded subsequence	Comparison	C > A	2	231	1.000
Bounded subsequence	Comparison	C > C	2	181	1.000
Bounded subsequence	Reference	A > C	2	106	1.000
Bounded subsequence	Reference	C > D	2	147	0.972
Bounded subsequence	Reference	D > A	2	144	0.972

The pattern results provide local structural evidence complementary to the whole-sequence distance analysis. Pattern prevalence is not interpreted as importance, mechanism, or causal influence.

3.6. Dissimilarity and Clustering Validation

Normalised Levenshtein analysis generated 2556 pairwise dissimilarities among the 72 sequences. Distances ranged from 0.292 to 0.833, with a mean of 0.600. The prespecified average-linkage k = 2 description yielded an average silhouette of 0.062, a minimum silhouette of −0.053, and a Dunn index of 0.526. These values indicate weak separation

under this representation; the revised manuscript therefore removes the earlier 'non-degenerate' characterisation (Figure 2).

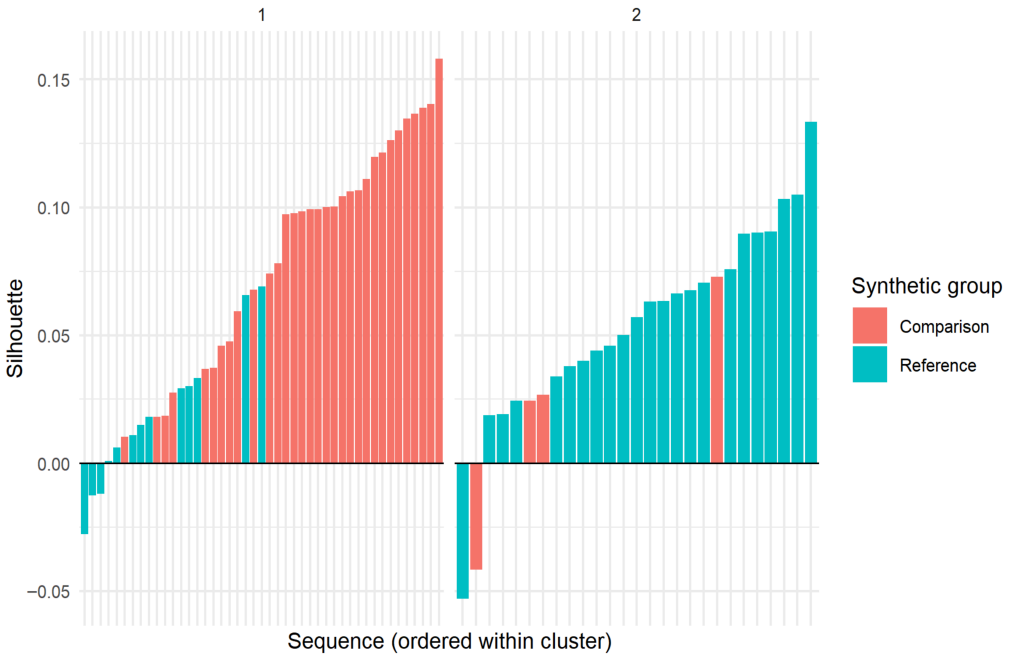


Figure 2. Per-sequence silhouette values for the prespecified two-cluster average-linkage description. Group labels are shown only to inspect composition; the clustering algorithm does not use them.

The clustering result is diagnostic rather than confirmatory. The generator differs primarily in transition tendencies, whereas normalised Levenshtein represents whole-path edit operations; weak silhouette values are therefore compatible with overlapping Markov support and do not contradict recovery of transition differences. Sensitivity checks across k, linkage, LCS and transition-profile dissimilarities show that internal separation and assignments vary with representation. Transition-sensitive distances can increase internal separation, but they do not supply a representation-independent 'true' partition. The 80% subsampling exercise likewise shows heterogeneous pairwise stability rather than a uniformly stable typology. These findings reinforce, rather than relax, the interpretation boundary around clustering.

3.7. Transition-Network Results

First-order transition networks were generated for each synthetic group and for the pooled data. The pooled centrality summary contained 4 state-level rows, corresponding to the observed four-state network. These quantities describe graph structure induced by observed transitions. They do not convert high degree, strength, closeness, or PageRank values into substantive attributes of the states.

The network stage demonstrates another representation change within the same workflow: canonical sequence data are converted into a transition graph while preserving the state alphabet and declared normalisation. This supports the cross-family aspect of RQ3 without implying that network analysis is interchangeable with whole-sequence dissimilarity.

3.8. Declared-Design Group Contrast

The synthetic observational comparison used sequence-level state-A prevalence. Comparison had mean prevalence 0.385 and Reference 0.265; under the recorded alphabetical group ordering, the contrast (Reference minus Comparison) was −0.120. With 999 unit-level label permutations, the two-sided p-value was 0.001. Resampling units with replacement within

group for 999 bootstrap replicates produced a 95% percentile interval of [−0.166, −0.073]. The resampling unit is the complete sequence, never an individual long-format row.

These values show whether the analysis recovers a deliberately introduced structural contrast in the current deterministic realisation. They are not estimates of a population parameter and are not causal effects. The design was declared observational, and the manuscript preserves that interpretation boundary.

3.9. Reproducibility and Software-Validation Evidence

Distance, clustering, and pooled transition-network objects returned audit status pass under the supported contract checks. Recomputing the distance object under identical inputs and settings remained numerically identical, while the added reference checks separately verified selected numerical semantics. This distinction is important: deterministic recomputation demonstrates repeatability, whereas agreement with base/specialist calculations addresses representative numerical correctness.

The benchmark-data MD5 is an execution-level content check rather than a universal identifier. The reproducibility anchor for the evaluated software is the immutable v0.3.0 tag and commit in Table 5, together with the CRAN package record and declared dependencies. A content-preserving software archive for an earlier release exists, but the present manuscript does not cite that older DOI as if it archived v0.3.0.

Table 5. Reproducibility and software-state evidence.

Evidence	Value
Evaluated gp3sequences version	0.3.0
Exact source tag	v0.3.0
Exact source commit	4ebf0bebea2955c5f98f8ddf0fe03e81d0b7ac3a
Persistent package identifier	CRAN DOI: 10.32614/CRAN.package.gp3sequences
Dependency specification	DESCRIPTION: Imports graphics, methods, stats, utils; optional specialist backends declared in Suggests
Primary synthetic-data seed	20260811
RNG specification	Mersenne-Twister/Inversion/Rejection
Benchmark data hash	34d2099346db7e9c4fe4a7b8c9dca908 (MD5; execution artefact)
Distance-analysis audit	pass
Clustering-analysis audit	pass
Transition-network audit	pass
Repeated distance computation	numerically identical
R version	R version 4.6.1 (24 June 2026 ucrt)
Operating system	Windows 10x64x86−64

3.10. Computational Scaling

The bounded scaling experiment evaluated 9 synthetic configurations, up to 50,000 long-format rows. Figure 3 shows median elapsed time for validation, preparation, state summaries, and transition summaries. Across the measured configurations, the maximum median operation time was 8.3100 s on the current machine. These timings are descriptive and should not be extrapolated to other hardware, package versions, sequence alphabets, or pairwise-distance workloads.

3.11. Additional Structural Visual Diagnostics

Complementary structural diagnostics are consolidated to reduce figure fragmentation. Figure 4 combines positional composition, observed transition probabilities, and motif prevalence; Figure 5 combines the dissimilarity matrix, sequence-level state-A prevalence, and the sequence-index display. These panels visualise objects already analysed above and introduce no additional estimands or tests.

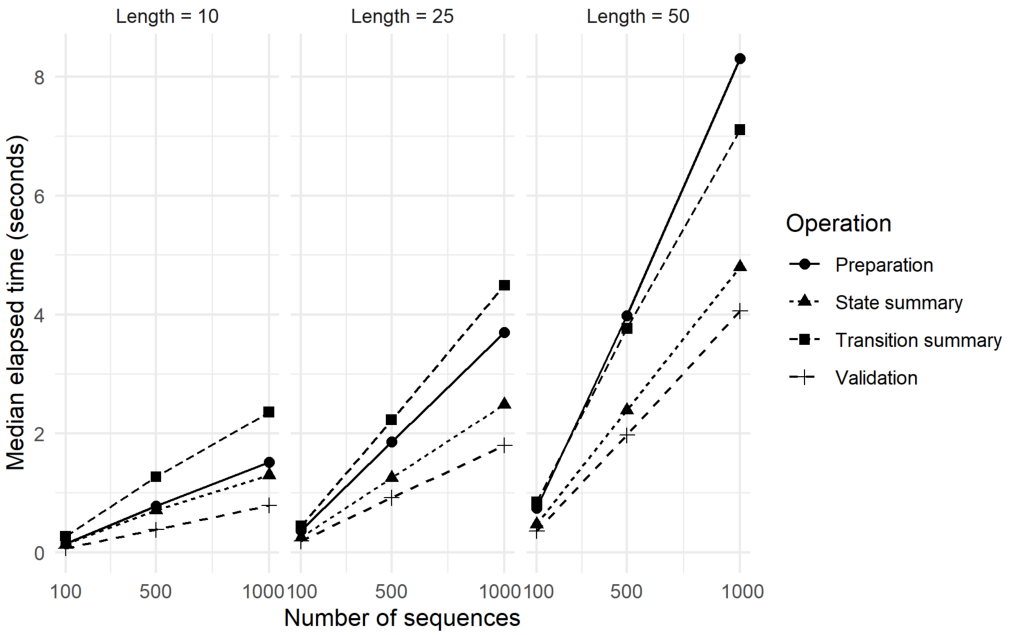


Figure 3. Environment-specific median elapsed time for validation, preparation, state summaries, and transition summaries across the bounded scaling design. Each operation is repeated three times per configuration; timings are local descriptive measurements, not cross-hardware performance guarantees.

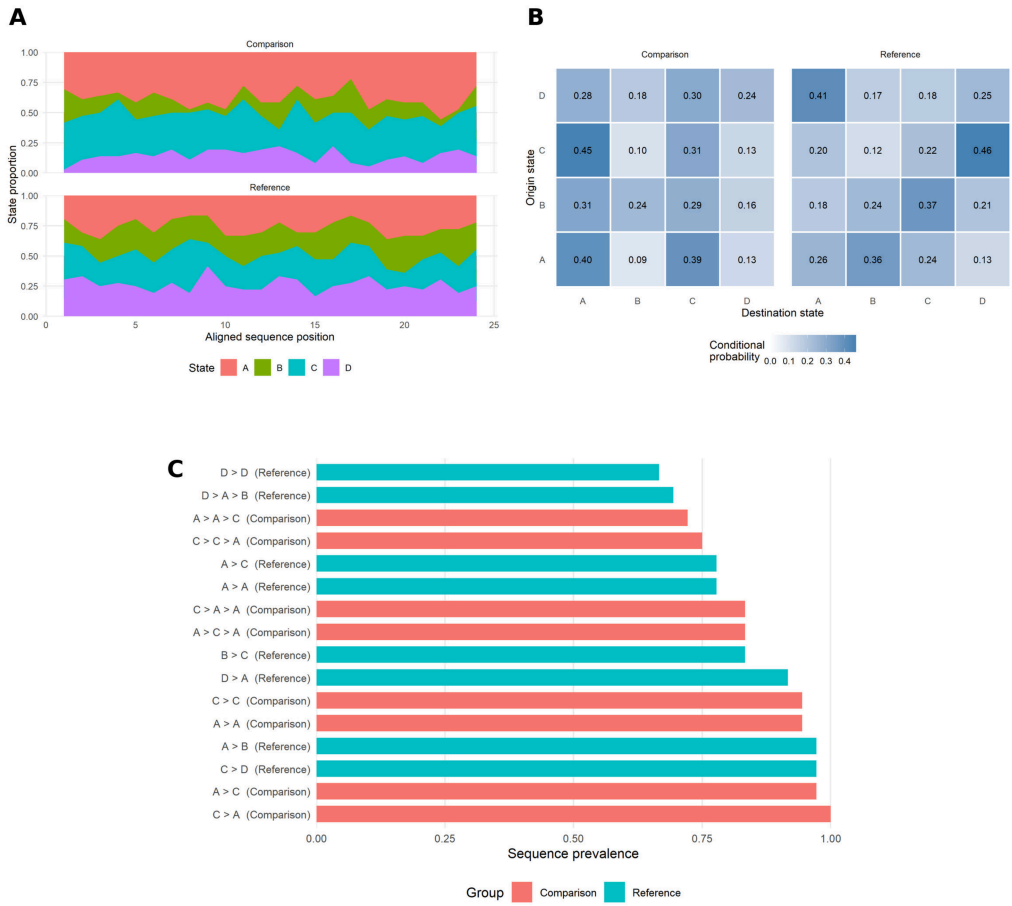


Figure 4. Consolidated structural diagnostics: (**A**) position-wise state composition by generating group; (**B**) observed first-order transition probabilities; and (**C**) sequence prevalence of common contiguous motifs. Panels describe realised structure and do not assign substantive meaning to state labels.

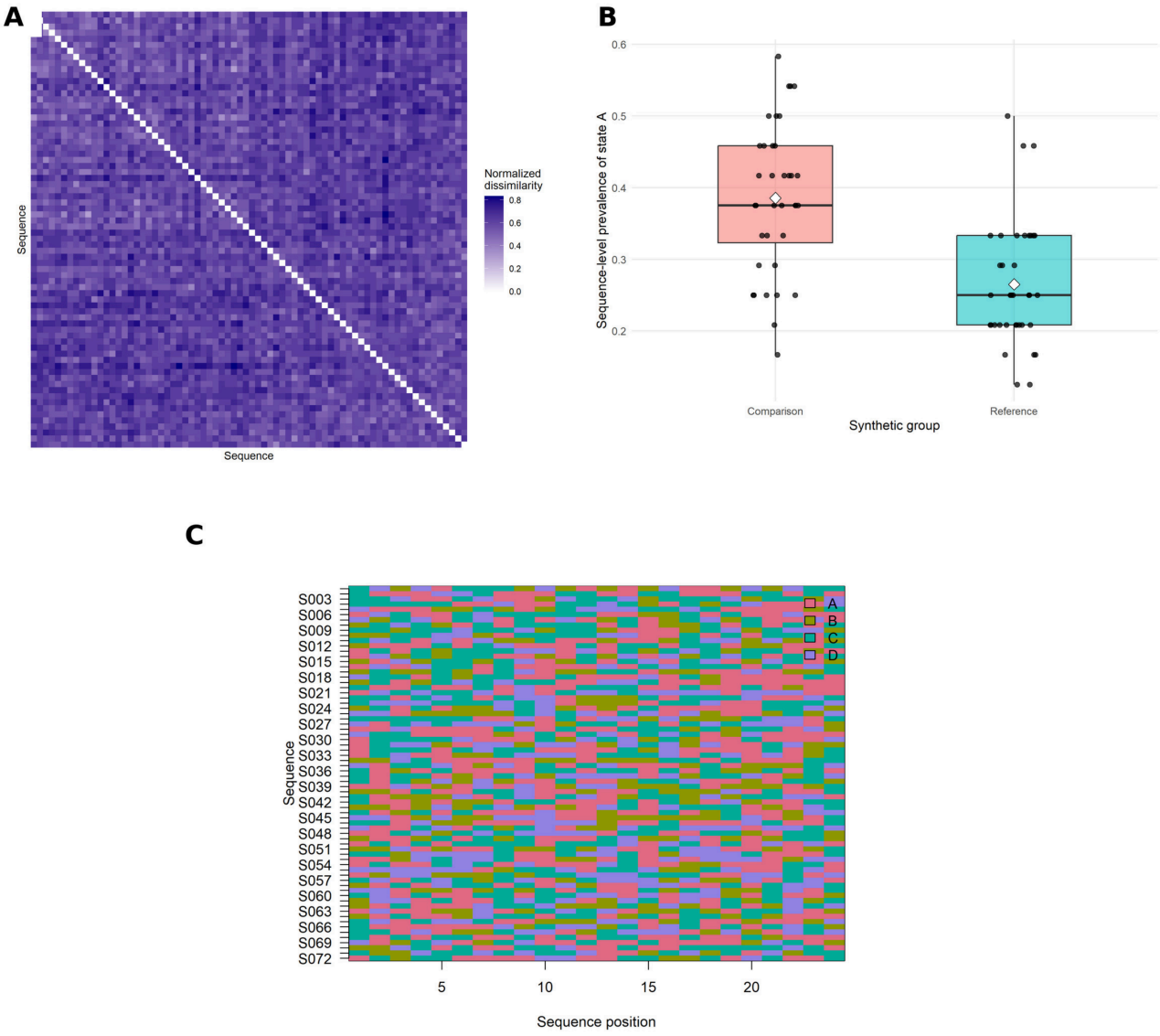


Figure 5. Consolidated whole-sequence diagnostics: (**A**) normalised Levenshtein dissimilarity matrix; (**B**) sequence-level prevalence of state A used in the declared observational contrast; and (**C**) sequence-index representation of the benchmark. Group ordering is used for display only.

3.12. Multi-Seed and Clustering Sensitivity

Across 100 additional seeds under the primary 72 × 24 generator, the deliberately introduced state-A contrast retained the same direction in every replicate; its magnitude varied, as expected for finite stochastic sequences. Smaller and shorter designs showed greater between-replication dispersion, whereas larger and longer designs reduced it. The six-state mvad example further shows that the data contract is not tied to a four-symbol alphabet. These checks support the robustness of the qualitative integration/recovery conclusions, not invariant effect sizes or universal power.

Clustering was deliberately less stable than the state and transition summaries. Across k = 2–5 and multiple linkages, Levenshtein-based silhouettes remained weak; LCS and transition-profile distances changed both internal separation and assignments. In the 80% subsampling analysis (200 resamples), pairwise agreement with the full-data k = 2 solution was heterogeneous rather than uniformly high. This sensitivity is consistent with the generator: the group contrast is transition-based, while edit-distance clustering represents a different aspect of the sequences. No clustering specification is selected as a preferred or ground-truth solution.

3.13. Independent Numerical-Reference Checks

Representative outputs from five analytical families were checked against independently expressed calculations or the underlying specialist/base implementation using identical inputs and settings. The checks are intentionally narrow: they establish numerical semantics for selected operations, not universal correctness of every package function. Table 6 reports the references and comparison criteria.

Table 6. Representative numerical-reference and contract checks.

Family	Reference Calculation	Comparison Criterion	Outcome
Dissimilarity	Base-R edit distance on selected sequence pairs, normalised by declared maximum length	absolute difference $\leq 1 \times 10^{-12}$	agreement
Motif frequency	Explicit sliding-window enumeration and sequence-level prevalence counts	exact integer counts	agreement
Transition probability	Base-R from/to contingency table with row-wise normalisation	absolute difference $\leq 1 \times 10^{-12}$	agreement
Clustering	Direct stats: hclust()/cutree(); cluster: silhouette() where available	assignments equal up to label permutation; diagnostics within tolerance	agreement
Permutation/ bootstrap inference	Direct recomputation from stored unit-level metrics and resampling distributions	estimate, corrected permutation *p*-value, and percentile interval reproduced	agreement
Analysis audit	Distance, clustering, and transition-network contract audits	required family/settings/identifier fields recoverable; no error issue	pass

The reference checks complement the repeated-computation test in Table 5. In particular, direct comparison to base/specialist calculations addresses the reviewer's distinction between determinism and numerical correctness.

3.14. Adversarial Failure Cases

The successful primary benchmark is complemented by deliberately invalid fixtures. Each fixture changes one structural property while holding the remaining input minimal and valid. The expected response is specified before execution. Table 7 shows that failures remain visible and policy-dependent rather than being repaired silently.

Table 7. Adversarial structural and dependency checks.

Adversarial Condition	Expected Diagnostic/Guard	Default Consequence in Test	Why It Is Not Silent
Duplicated sequence position	duplicated_ position; error	preparation fails under duplicate_position _policy = "error"	conflicting order is returned with sequence/ row context
Missing state	missing_state; error	preparation fails under missing_state _policy = "error"	row is not coerced to a state or zero

Table 7. *Cont.*

Adversarial Condition	Expected Diagnostic/Guard	Default Consequence in Test	Why It Is Not Silent
Undeclared symbol	unknown_state; review at audit	explicit error/preserve/drop policy required; refusal used in test	state-alphabet mismatch remains visible
Inconsistent metadata within sequence	inconsistent_ metadata; error	preparation fails	sequence identity/metadata conflict must be corrected or sequence redefined
Unavailable optional backend	dependency guard in adapter	adapter stops with package-specific requirement	optional package is not installed or substituted silently

The adversarial cases operationalise the framework's failure contract. They do not show that every possible malformed dataset is anticipated; they demonstrate that the specific structural problems central to the manuscript have explicit diagnostic paths.

3.15. Public Empirical Hand-Off

The mvad worked example exercises the same contract with a larger six-state alphabet and real metadata. Reshaping the 712 individuals $\times$ 72 monthly positions yields an ordinary long representation while retaining identifiers, sample weights, and selected covariates. Conversion through as_traminer_sequences() preserves sequence identifiers as external row names and the declared six-state alphabet. This is direct evidence for the adapter boundary claimed in Section 2.6. The example is intentionally non-inferential: it demonstrates mapping and interoperability under a public empirical structure without presenting new substantive conclusions about the original cohort.

4. Discussion

4.1. Principal Findings

The revised evaluation provides qualified answers to RQ1–RQ5 while making the evidential boundaries more explicit. RQ1 is supported by the common long-format contract together with both successful and adversarial preparation cases. RQ2 is supported across distance, clustering, and transition-network audit objects and by representative numerical-reference checks; it is not claimed for every public function. RQ3 is supported by the primary benchmark plus multi-seed and dimension sensitivity, while the weak and representation-dependent clustering result is treated as a limitation rather than as evidence of latent classes. RQ4 is addressed through a versioned, source-backed comparison and a concrete TraMineR hand-off. RQ5 is restricted to the four lightweight operations actually timed.

The main architectural result is therefore not a new sequence estimator. It is the explicit binding of sequence-specific mappings, preparation decisions, analytical settings, capability state, diagnostics, and provenance to objects that cross otherwise heterogeneous analytical families. Generic literate-programming tools can record code that performs these tasks, but they do not themselves impose these sequence-specific contracts.

The sensitivity analyses also change the interpretation of the synthetic benchmark. The fixed seed remains necessary for exact manuscript regeneration, but the structural contrast is not presented as credible merely because one realisation recovered it. Replicated seeds show directional stability for the deliberately introduced state contrast, whereas the

clustering analyses remain weak and representation-sensitive. This contrast is informative: different sequence representations preserve different aspects of the same stochastic paths.

The specialist-software comparison should likewise be read as a division of methodological ownership, not as evidence that gp3sequences is more complete, accurate, or efficient. The added empirical hand-off provides direct evidence that at least one such boundary can preserve identifiers and alphabet semantics in practice; other adapters remain subject to their specialist package contracts and version-specific behaviour.

4.2. Scientific-Software Contribution

The software contribution is a domain-specific governance layer that makes analytical state inspectable at runtime. A final statistic alone does not specify a sequence workflow: column roles, ordering, state alphabets, preparation policies, distance settings, model options, resampling units, seeds, optional backends, and software versions can all alter meaning or reproducibility. gp3sequences structures these items as mappings, decision logs, analysis contracts, capability records, and provenance fields. This is more concrete than claiming reproducibility as a general property, but narrower than claiming a new statistical paradigm [16,17,19].

The failure contract is equally important. Duplicate positions and inconsistent metadata are errors; missing or undeclared states require explicit policy; optional backends stop with an explicit dependency requirement. The adversarial examples make these behaviours observable. The framework cannot determine which policy is scientifically correct for a given study; it can require that the choice be explicit and inspectable.

A Python implementation, gp3sequencespy, is under development as future work and is outside the scope of the present evaluation.

4.3. Relationship to Specialist Software

The comparison should not be read as an argument for centralising sequence analysis in one package. TraMineR's sequence-object semantics and mature state-sequence methods remain valuable when the scientific question is centred on that paradigm [2]. seqHMM offers a direct specialist environment for hidden and mixture hidden Markov modelling [8]. WeightedCluster, ClickClust, PST, arulesSequences, and march likewise serve focused methodological roles [9,10,12–14].

The useful boundary is therefore methodological ownership. gp3sequences can standardise validation, preparation, settings, and audit around a multi-stage workflow, but it should delegate when a specialist object or specialised diagnostic is scientifically preferable. Guarded interoperability is valuable precisely because it allows such delegation without representing every external capability as native.

4.4. Reproducibility Implications

Reproducibility is accumulated during computation rather than added only at submission. The revised manuscript pins the evaluated package to tag v0.3.0 and commit 4ebf0bebea2955c5f98f8ddf0fe03e81d0b7ac3a, reports the dependency specification and runtime environment, and distinguishes the synthetic-data content hash from the software identifier. These measures reduce ambiguity about the evaluated code while leaving specialist optional dependencies explicitly version-sensitive.

This approach is compatible with research-compendium and FAIR principles but does not subsume them [19,20]. Durable reuse still requires archived source, executable manuscript code, dependency availability, and adequate documentation. The contract layer addresses the narrower problem of preserving analytical state inside the workflow [19,20].

4.5. Interpretation Governance

Interpretation governance is applied once across analytical families: structural outputs inherit only the meaning warranted by their mathematical representation and study design. Thus, clustering describes a partition under a specified distance/linkage, latent states remain model states, network measures describe transition graphs, and permutation contrasts inherit the declared resampling design. Substantive or causal interpretation requires evidence external to these software objects.

This boundary is a software safeguard rather than a methodological finding; it is retained in result metadata and documentation without repeating the same caution after every analysis.

5. Limitations

Several limitations bound the contribution. First, the primary performance and recovery evaluation remains synthetic. The added mvad example improves ecological realism for data structure and interoperability, but it is deliberately not a substantive external-validation study. Domain applications may require different state definitions, time scales, missingness policies, dependence assumptions, or sampling structures.

Second, the numerical-reference checks are representative rather than exhaustive. Agreement for selected distances, motifs, transition probabilities, clustering operations, and resampling summaries does not certify every estimator or optional backend. Specialist comparisons are scope comparisons, not head-to-head claims of accuracy or speed.

Third, integration introduces conventions of its own. Canonical column names, result contracts, adapter boundaries, and diagnostic statuses are design choices that users must understand. Auditability reduces hidden state but cannot eliminate methodological judgement.

Fourth, sensitivity analyses are bounded. The seed and dimension grid reduces dependence on one stochastic realisation, but it does not span arbitrary alphabet sizes, higher-order generators, rare-state regimes, or all transition contrasts. The public six-state example addresses one larger alphabet only. Cluster stability is likewise conditional on the chosen distance, linkage, k, and resampling scheme.

Fifth, comparator evidence is version- and documentation-dependent. The table records the versions and access date reviewed; 'not documented' is not interpreted as 'not possible', and package capabilities may change.

Sixth, scalability evidence is deliberately narrow and RQ5 has been rewritten accordingly. Timings cover only validation, preparation, state summaries, and transition summaries, over at most 50,000 rows on one environment. They do not establish memory complexity or performance for pairwise distances, clustering, pattern mining, or HMM fitting.

Finally, optional dependencies create heterogeneous computational environments. Guarded adapters make that heterogeneity explicit, but numerical parity with specialist backends still requires matching method definitions and software versions.

6. Conclusions

This study evaluates gp3sequences as a contract-oriented scientific-software architecture for ordered categorical sequence analysis. The evidence supports a constrained contribution: sequence-specific mappings, preparation policies, analytical settings, diagnostics, dependency state, and provenance can be kept inspectable while a workflow crosses several structural analysis families and selected specialist boundaries. Independent numerical checks, adversarial fixtures, seed/dimension sensitivity, and a public TraMineR hand-off strengthen the evaluation beyond a single successful synthetic execution.

The framework remains complementary to specialist sequence software and to general reproducible-computing tools. The revised results do not support claims of universal

clustering structure, methodological superiority, or broad empirical validity. They support the narrower proposition that explicit contracts and guarded hand-offs can make heterogeneous sequence workflows easier to audit, reproduce, and challenge.

Funding: This research received no external funding.

Data Availability Statement: No new empirical data were collected. The primary benchmark is generated synthetically by the manuscript workflow. The public mvad example is distributed with TraMineR and originates from McVicar and Anyadike-Danes [22]. gp3sequences source code and documentation are available at https://github.com/stefanosbalaskas/gp3sequences (accessed on 21 September 2026) and https://CRAN.R-project.org/package=gp3sequences (accessed on 21 September 2026). The evaluated software is pinned to tag v0.3.0, commit 4ebf0bebea2955c5f9-8f8ddf0fe03e81d0b7ac3a; the CRAN package DOI is https://doi.org/10.32614/CRAN.package.gp3sequences. The manuscript-generating R Markdown and revision-specific analysis outputs are provided at https://github.com/stefanosbalaskas/gp3sequences (accessed on 21 September 2026).

Conflicts of Interest: The author declares no conflicts of interest.

Appendix A

The original workflow schematic is retained as supplementary architectural orientation. The feature-level distinction between general reproducible workflow practice and gp3sequences is reported in Figure A1.

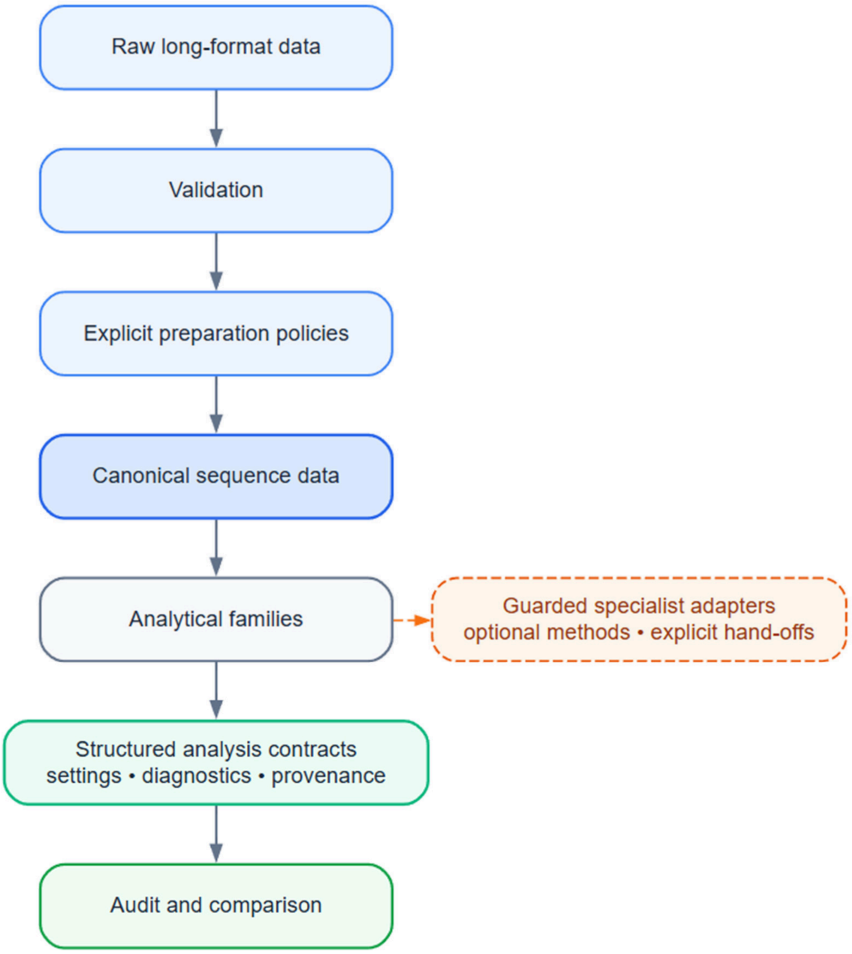


Figure A1. Contract-oriented workflow architecture of gp3sequences. The schematic separates the persistent long-format data contract, explicit preparation, analytical modules, specialist delegation, and the audit/provenance layer.

References

1. Abbott, A.; Tsay, A. Sequence Analysis and Optimal Matching Methods in Sociology: Review and Prospect. *Sociol. Methods Res.* **2000**, *29*, 3–33. [CrossRef]
2. Gabadinho, A.; Ritschard, G.; Müller, N.S.; Studer, M. Analyzing and Visualizing State Sequences in R with TraMineR. *J. Stat. Softw.* **2011**, *40*, 1–37. [CrossRef]
3. Studer, M.; Ritschard, G. What Matters in Differences Between Life Trajectories: A Comparative Review of Sequence Dissimilarity Measures. *J. R. Stat. Soc. Ser. A Stat. Soc.* **2016**, *179*, 481–511. [CrossRef]
4. Ritschard, G.; Studer, M. Sequence Analysis: Where Are We, Where Are We Going? In *Sequence Analysis and Related Approaches: Innovative Methods and Applications*; Life Course Research and Social Policies; Ritschard, G., Studer, M., Eds.; Springer: Berlin/Heidelberg, Germany, 2018; Volume 10. [CrossRef]
5. Zaki, M.J. SPADE: An Efficient Algorithm for Mining Frequent Sequences. *Mach. Learn.* **2001**, *42*, 31–60. [CrossRef]
6. Rabiner, L.R. A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition. *Proc. IEEE* **1989**, *77*, 257–286. [CrossRef] [PubMed]
7. Rousseeuw, P.J. Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *J. Comput. Appl. Math.* **1987**, *20*, 53–65. [CrossRef]
8. Helske, S.; Helske, J. Mixture Hidden Markov Models for Sequence Data: The seqHMM Package in R. *J. Stat. Softw.* **2019**, *88*, 1–32. [CrossRef]
9. Buchta, C.; Hahsler, M. *ArulesSequences: Mining Frequent Sequences*; CRAN: Vienna, Austria, 2025. [CrossRef]
10. Matthias, S. *WeightedCluster Library Manual: A Practical Guide to Creating Typologies of Trajectories in the Social Sciences with R*; LIVES Working Papers No. 24. NCCR LIVES; University of Lausanne: Vaud, Switzerland, 2013. [CrossRef]
11. Studer, M.; Roth, L. *WeightedCluster: Clustering of Weighted Data*; CRAN: Vienna, Austria, 2025. [CrossRef]
12. Melnykov, V. ClickClust: An R Package for Model-Based Clustering of Categorical Sequences. *J. Stat. Softw.* **2016**, *74*, 1–34. [CrossRef]
13. Gabadinho, A.; Ritschard, G. Analyzing State Sequences with Probabilistic Suffix Trees: The PST R Package. *J. Stat. Softw.* **2016**, *72*, 1–39. [CrossRef]
14. Berchtold, A.; Maitre, O.; Emery, K. Optimization of the Mixture Transition Distribution Model Using the March Package for r. *Symmetry* **2020**, *12*, 2031. [CrossRef]
15. Maitre, O.; Emery, K.; Buschor, O.; Berchtold, A. *March: Markov Chains*; CRAN: Vienna, Austria, 2020. [CrossRef]
16. Sandve, G.K.; Nekrutenko, A.; Taylor, J.; Hovig, E. Ten Simple Rules for Reproducible Computational Research. *PLoS Comput. Biol.* **2013**, *9*, e1003285. [CrossRef] [PubMed]
17. Wilson, G.; Aruliah, D.A.; Brown, C.T.; Hong, N.P.C.; Davis, M.; Guy, R.T.; Haddock, S.H.D.; Huff, K.D.; Mitchell, I.M.; Plumbley, M.D.; et al. Best Practices for Scientific Computing. *PLoS Biol.* **2014**, *12*, e1001745. [CrossRef] [PubMed]
18. Taschuk, M.; Wilson, G. Ten Simple Rules for Making Research Software More Robust. *PLoS Comput. Biol.* **2017**, *13*, e1005412. [CrossRef] [PubMed]
19. Marwick, B.; Boettiger, C.; Mullen, L. Packaging Data Analytical Work Reproducibly Using R (and Friends). *Am. Stat.* **2018**, *72*, 80–88. [CrossRef]
20. Wilkinson, M.D.; Dumontier, M.; Aalbersberg, I.J.; Appleton, G.; Axton, M.; Baak, A.; Blomberg, N.; Boiten, J.W.; da Silva Santos, L.B.; Bourne, P.E.; et al. The FAIR Guiding Principles for Scientific Data Management and Stewardship. *Sci. Data* **2016**, *3*, 160018. [CrossRef] [PubMed]
21. Balaskas, S. *Gp3sequences: Transparent Analysis of Ordered Categorical Sequences*; Version 0.3.0; CRAN Package: Vienna, Austria, 2026; Source Tag v0.3.0, Commit 4ebf0bebea2955c5f98f8ddf0fe03e81d0b7ac3a. Available online: https://github.com/stefanosbalaskas/gp3sequences (accessed on 21 September 2026). [CrossRef]
22. McVicar, D.; Anyadike-Danes, M. Predicting Successful and Unsuccessful Transitions from School to Work by Using Sequence Methods. *J. R. Stat. Soc. Ser. A Stat. Soc.* **2002**, *165*, 317–334. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.